



SESIÓN 3: Buenas prácticas REST + Robustez

"La API funciona... pero es frágil"



OBJETIVO REAL DE LA SESIÓN

Después de la sesión 2:

- La API funciona
- Guarda datos en MySQL
- Pero...



👉 Problemas reales:

- Devuelve null
- No comunica errores correctamente
- El cliente no sabe qué ha pasado

Objetivo de hoy: 👉 Convertir una API "que funciona" en una API **robusta y profesional**.



TEORÍA CLAVE (muy importante)

1

Una API no comunica con null

En backend:

- null **no es información**
- El cliente no sabe si:
 - no existe
 - hubo error
 - está mal la petición



Mensaje clave de la sesión:

"Una API profesional comunica con códigos, no con nulls."

2 HTTP Status Codes esenciales (los justos)

No enseñar todos. Solo los que **van a usar**:

Código	Cuándo
200 OK	Lectura correcta
201 CREATED	Recurso creado
204 NO CONTENT	Borrado correcto
400 BAD REQUEST	Petición inválida
404 NOT FOUND	Recurso no existe



💡 Importante:

El **status** es tan importante como el **body**.

3 ResponseEntity

ResponseEntity permite controlar:

status code

body

headers

👉 Sin ResponseEntity, Spring **asume 200 siempre**.



OBJETIVO DE REFACTORIZACIÓN

Vamos a transformar esto:

```
return bookRepository.findById(id).orElse(null);
```

en esto:

```
return bookRepository.findById(id)
    .map(ResponseEntity::ok)
    .orElse(ResponseEntity.notFound().build());
```



BLOQUE 1 — Refactor: GET by id



controller/BookController.java (fragmento)

```
@GetMapping("/{id}")
public ResponseEntity<Book> getById(@PathVariable Long id) {
    return bookRepository.findById(id)
        .map(ResponseEntity::ok)    // 200 + body
        .orElse(ResponseEntity.notFound() // 404 sin body
            .build());
}
```

Aquí



Optional fuerza a pensar el caso "no existe"



map → solo si hay valor



orElse → caso vacío



404 comunica perfectamente el problema



Mensaje clave:

"El cliente ahora **entiende** qué ha pasado."

BLOQUE 2 — GET all con 200 explícito

```
@GetMapping
public ResponseEntity<List<Book>> getAll() {
    return ResponseEntity.ok(bookRepository.findAll());
}
```

 Por qué hacerlo:

Claridad

El código expresa intención explícita

Coherencia

Todos los endpoints usan el mismo patrón

Control explícito del status

No dependemos de comportamiento por defecto

BLOQUE 3 — POST correcto (201 CREATED)

Antes (incorrecto REST)

```
@PostMapping
public Book create(@RequestBody Book book) {
    return bookRepository.save(book);
}
```

Después (REST correcto)

```
@PostMapping
public ResponseEntity<Book> create(@RequestBody Book book) {
    Book saved = bookRepository.save(book);
    return ResponseEntity
        .status(201) // CREATED
        .body(saved);
}
```

Explicación

1

POST **crea** algo nuevo

2

200 no es incorrecto, pero **201 es correcto**

3

El status comunica intención

 Mensaje clave:

"REST no es solo que funcione, es que **comunique bien**."

BLOQUE 4 — PUT robusto

 BookController.java

```
@PutMapping("/{id}")
public ResponseEntity<Book> update(
    @PathVariable Long id,
    @RequestBody Book updated) {

    return bookRepository.findById(id)
        .map(existing -> {
            existing.setTitle(updated.getTitle());
            existing.setAuthor(updated.getAuthor());
            existing.setCategory(updated.getCategory());
            Book saved = bookRepository.save(existing);
            return ResponseEntity.ok(saved); // 200
        })
        .orElse(ResponseEntity.notFound().build()); // 404
}
```

Qué remarcar

PUT **no crea**, actualiza

Si no existe → 404

Nunca devolver null

BLOQUE 5 — DELETE correcto (204 NO CONTENT)

 BookController.java

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    if (!bookRepository.existsById(id)) {
        return ResponseEntity.notFound().build(); // 404
    }
    bookRepository.deleteById(id);
    return ResponseEntity.noContent().build(); // 204
}
```

Explicación clave

DELETE exitoso no devuelve body

204 = "todo bien, no hay nada que devolver"

 Mensaje clave:

"No todo endpoint tiene que devolver JSON."



BLOQUE 6 — Controller completo (foto final)

```
@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookRepository bookRepository;

    public BookController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @GetMapping
    public ResponseEntity<List<Book>> getAll() {
        return ResponseEntity.ok(bookRepository.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getById(@PathVariable Long id) {
        return bookRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Book> create(@RequestBody Book book) {
        Book saved = bookRepository.save(book);
        return ResponseEntity.status(201).body(saved);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Book> update(
        @PathVariable Long id,
        @RequestBody Book updated) {

        return bookRepository.findById(id)
            .map(existing -> {
                existing.setTitle(updated.getTitle());
                existing.setAuthor(updated.getAuthor());
                existing.setCategory(updated.getCategory());
                return ResponseEntity.ok(bookRepository.save(existing));
            })
            .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        if (!bookRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        bookRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```



BLOQUE 7 — Pruebas en Swagger

Pruebas:

01

GET id inexistente → **404**

02

POST → **201**

03

DELETE → **204**

04

PUT id inexistente → **404**



CIERRE DE LA SESIÓN

Qué sabes:

✓ HTTP Status Codes

✓ ResponseEntity

✓ Optional bien usado

✓ API robusta

✓ No devolver null

"Una API profesional comunica con códigos, no con nulls."